2회차 강사 큐시트
원천의 변경이 어떤 이벤트로 수집되는가

120분 수업용이다. 실행은 강사 맥미니에서 하고 학생은 SQL·이벤트 결과를 예측하며 강사 화면과 비교한다. 화면 조회는 `--summary --evidence`로 짧게 보여주며 전체 원문은 증거 JSON에 보존한다. 원문 key/value·payload가 필요하면 같은 조회에서 `--summary`만 제외한다. 개인 환경에서 직접 실행하는 복습과 수업 중 활동을 구분한다. 진행표는 강사 진행안, 기술 근거는 공식 참고 문서를 함께 본다.

수업 전 — 화면 공유 전에 완료

실행 위치: 강사 노트북 터미널 → SSH 접속 후 맥미니 저장소 루트. 이미 환경이 기동되어 있다면 무조건 재기동하지 말고 상태부터 확인한다. 이미지 빌드·pull, connector 재등록 실험, 슬롯·토픽·볼륨 초기화는 수업 중 하지 않는다.

```
ssh mini
cd ~/bootcamp/cdc-lakehouse-study
./scripts/lab.sh ps
./scripts/lab.sh exec -T runner python labs/session01/preflight.py
```

환경이 정지되어 있을 때의 기동은 기존 실행 안내를 따른다. 준비된 로컬 이미지로 `bootstrap.sh --no-build`를 사용하고, preflight의 `ready: true`, 최근 완료 checkpoint와 `checkpoint_failures_during_check: 0`을 확인한다. 기동 직후 누적 Failed만 보고 성공·실패를 단정하지 않는다. bootstrap의 준비용 변환을 오늘 수집 시연과 구분한다.

오늘 읽을 이력 선택: 아래는 1회차 수업의 환불 키다. 과거에 발생한 변경을 현재 테이블과 실제 파일에서 다시 읽는 것이며, 오늘 새로 만든 이벤트가 아니다.

```
CASE_ID=s01-live-10181152
printf '오늘 확인할 환불 ID: %s-r1\n' "$CASE_ID"
./labs/session02/lab.sh "$CASE_ID" source-rows --evidence
./labs/session02/lab.sh "$CASE_ID" bronze --summary --evidence
./labs/session02/lab.sh "$CASE_ID" parquet --summary --evidence
```

원천 0행과 해당 키의 c/u/d 세 이력, 실제 Parquet 디코드 결과까지 미리 확인한다. 기존 수업 키가 없으면 맨 아래의 수업 전 대체 예제 준비를 사용하고, 오늘은 새 복습용 키를 읽는다고 안내한다. 같은 키의 데이터를 지워서 준비하지 않는다. 보존된 키가 있는데 예상과 다르면 원인을 기록하고 임의로 고치지 않는다.

독립 파서의 예시 입력과 테스트도 미리 확인한다. 이 명령은 원천 DB나 Kafka를 변경하지 않는다.

```
python3 labs/session02/event_parser.py --fixtures
python3 -m unittest discover -s labs/session02/tests -v
```

Flink UI를 함께 보려면 강사 노트북의 별도 로컬 터미널에서 터널을 연다.

```
ssh -N -L 28081:127.0.0.1:28081 mini
```

노트북 브라우저는 `http://localhost:28081`을 사용한다. 학생은 SSH 접속이 필요 없다. `.env`, 비밀번호가 들어간 SQL·전체 connector 설정은 화면에 띄우지 않는다.

00:00–00:10 — 진행 공유와 오늘 볼 결과

화면: 공통 예제 흐름과 오늘 시간표. 처음 5분에는 두 사람 정도의 진행 상황을 듣는다.

강사 멘트

> “지난 시간 뒤에 어디까지 실행해보셨나요? 환경이 떴는지, 환불 변경이 어디까지 보였는지, 막혔다면 어느 명령이었는지 한 가지만 얘기해주세요. 아직 끝내지 못하셨어도 괜찮습니다. 오늘 어느 구간부터 확인하면 될지 같이 연결해볼게요.”

> “지난번에는 환불을 2만 원으로 만들고, 3만 원으로 바꾼 뒤 삭제했습니다. 원천에서는 사라졌는데 Bronze에는 세 이력이 남는다고 했죠. 오늘은 그 결과를 SELECT와 실제 파일 내용으로 먼저 확인하겠습니다.”

학생에게 질문: “지금 원천에서는 몇 행, Bronze에서는 어떤 세 이벤트를 예상하시나요?” 답은 구두나 채팅으로 받고 별도 표를 작성시키지 않는다. 진행 공유에서 나온 설치 문제는 따로 남겨두고 전체 수업을 멈춰 해결하지 않는다.

00:10–00:30 — 원천 0행, Bronze 세 이력, 실제 Parquet

실행 위치: 맥미니 저장소 루트, 같은 터미널. 수업 전에 정한 CASE_ID를 유지한다.

```
printf '현재 비교할 환불 ID: %s-r1\n' "$CASE_ID"
./labs/session02/lab.sh "$CASE_ID" source-rows --evidence
./labs/session02/lab.sh "$CASE_ID" bronze --summary --evidence
```

예상 관찰: 원천의 해당 환불은 0행이다. Bronze의 `record_count=3`, `history_ops=["c","u","d"]`를 확인하고 금액은 null→20000, 20000→30000, 30000→null로 읽는다. `(topic, partition_id, kafka_offset)`과 `parquet_file`도 함께 본다. 같은 토픽·파티션 안의 수집 위치로 정렬하며 숫자를 외우게 하지 않는다.

강사 멘트

> “원천에서는 이제 이 환불을 찾을 수 없습니다. 그런데 Bronze에는 처음 생성한 값, 정정한 값, 삭제한 이벤트가 따로 남아 있네요. 지금 세 행은 테이블 전체가 세 행이라는 뜻이 아니라, 오늘 고른 환불 ID의 이력입니다.”

> “삭제 이벤트는 after가 null입니다. after에서만 환불 ID를 찾는 SELECT를 쓰면 마지막 행을 놓치겠죠. 이 행을 찾으려면 before도 봐야 합니다.”

학생에게 질문: “조회 조건을 after의 refund_id에만 걸었다면 c/u/d 중 무엇이 빠질까요?” 학생 SQL의 조건과 비교하고 결과 화면을 잠시 고정한다.

이제 테이블 조회를 실제 파일과 연결한다.

```
./labs/session02/lab.sh "$CASE_ID" files --summary --evidence
./labs/session02/lab.sh "$CASE_ID" parquet --summary --evidence
```

예상 관찰: `files`는 이번 키의 SQL 행과 연결된 경로를 현재 files 메타데이터에서 확인한다. `files`의 `file_path`, `file_format`, `record_count`, `file_size_in_bytes`를 읽는다. `parquet`에서는 `physical_files` 안의 `byte_count`, `sha256`, `magic_start`, `magic_end`, `decoded_record_count`, `matching_records`를 확인한다. 상위 결과의 `passed=true`, `sql_file_records_match=true`, `matching_record_count=3`, `three_event_history_complete=true`까지 본다. 해시와 PAR1만이 아니라 디코드한 실제 행이 일치하는지 확인한다. 파일 수나 해시는 리허설 값으로 고정하지 않는다.

강사 멘트

> “앞의 SELECT는 Iceberg 테이블을 읽은 결과고, 지금은 그 결과와 연결된 Parquet 파일 내용을 읽고 있습니다. 파일 이름만 보이는 것과 파일 안의 행을 확인하는 건 다르죠. 방금 본 환불 ID와 이벤트가 실제 내용에도 있는지 대조해보겠습니다.”

> “테이블 하나가 파일 하나라는 뜻은 아닙니다. 지금 c, u, d가 파일 몇 개에 나뉘어 있는지는 출력대로 확인하세요. 일반적인 조회에서는 Iceberg가 현재 snapshot에 필요한 파일을 골라 읽습니다. 오늘 파일 직접 읽기는 내용 확인을 위한 것입니다.”

강사 참고: 물리 파일 읽기는 helper가 Spark 컨테이너 안의 Iceberg `S3FileIO`와 `GenericParquetReaders`로 수행한다. Spark SQL로 테이블을 한 번 더 조회하고 파일을 읽었다고 이름만 바꾸는 방식이 아니다. 수업에서는 Java 구현 자체를 강의하지 않고 경로·실제 행의 연결에 집중한다.

공식 근거: Iceberg 1.10 현재 파일 조회, S3FileIO, GenericParquetReaders, Parquet 파일 형식.

막히면: 원천만 확인됐는지, Bronze 조회까지 됐는지, 파일 경로 선택 또는 실제 파일 읽기에서 실패했는지 구분한다. 파일 하나를 임의로 열어 같은 데이터라고 설명하지 않는다. 5분을 넘기면 실행 시각과 키가 표시된 리허설의 실제 디코드 결과로 전환하고 과거 결과임을 밝힌다.

00:30–00:50 — PostgreSQL과 Debezium의 수집 조건

```
./labs/session02/lab.sh source-config --evidence
```

화면에서 짚을 순서: `wal_level` → publication의 테이블 → refunds의 replica identity → slot 상태. 조회만 수행하며 수집 설정을 변경하지 않는다.

강사 멘트

> “Flink가 DB 테이블을 계속 SELECT해서 가져오는 구조는 아닙니다. PostgreSQL이 WAL에 남긴 변경을 논리 변경으로 읽고, Debezium이 Kafka 이벤트로 보냅니다. 다만 WAL이 있다는 것만으로 모든 테이블을 자동 수집하는 건 아닙니다. 지금 어떤 테이블이 publication에 들어 있는지부터 보겠습니다.”

> “지난번 UPDATE에서 이전 금액 2만 원도 보였죠. 이 예제는 refunds에 REPLICA IDENTITY FULL을 설정해두었습니다. 이전 값을 얼마나 남길지는 원천 쪽 조건이므로 이벤트를 읽기 전에 확인해야 합니다.”

학생에게 질문: “before 금액이 안 보이면 파서부터 고치면 될까요? 먼저 어느 설정을 볼까요?” FULL의 목적을 답하고, 수업 중 기존 테이블 설정을 바꾸지 않는다.

slot에서는 `active`, `restart_lsn`, `confirmed_flush_lsn`을 나란히 본다.

> “슬롯은 소비자가 다시 이어 읽는 데 필요한 위치와 WAL 보존에 관련된 상태를 갖고 있습니다. 지금 위치를 보는 SELECT만 하겠습니다. 이 슬롯을 다른 프로그램으로 읽거나 위치를 앞으로 보내는 실험은 하지 않습니다.”

> “이 값은 초 단위 지연이 아닙니다. 또 슬롯 소비가 진행됐다는 것과 Iceberg에서 조회된다는 것도 같은 말은 아니죠. Kafka, Flink, Iceberg의 결과는 각 구간에서 따로 확인해야 합니다.”

이어 Debezium이 같은 원천을 어떻게 읽는지 필요한 설정만 본다.

```
./labs/session02/lab.sh connector-sanitized --evidence
```

화면에서 짚을 항목: plugin `pgoutput`, slot·publication 이름, include list, topic prefix, snapshot mode, tombstone 설정. 전체 connector 설정을 직접 출력하지 않는다.

> “원천 publication에서 내보낼 테이블을 봤다면, 여기서는 Debezium이 그중 무엇을 읽을지 봅니다. slot과 publication 이름, include list를 앞의 원천 설정과 연결해보세요.”

학생에게 질문: “publication에는 있는데 include list에 빠져 있다면 수집될까요?” 코드와 설정을 근거로 답한다. Debezium 3.2 PostgreSQL 설정.

공식 근거: PostgreSQL 17 Logical Decoding, Publication, REPLICA IDENTITY, slot 상태 컬럼.

막히면: 권한 오류나 예상과 다른 설정을 기록한다. 기존 slot·publication을 삭제하거나 새로 생성하지 않는다. 설정 의미는 준비한 안전한 출력으로 설명하고 00:50에 휴식한다.

00:50–00:55 — 휴식

> “5분 쉬고 돌아와서, 이 설정이 Kafka에서 어떤 이벤트가 되는지 이어서 보겠습니다.”

00:55–01:15 — snapshot 비교와 작은 파서

실행 위치: 맥미니 저장소 루트. 먼저 r 표본이 어디서 왔는지 확인한다.

```
./labs/session02/lab.sh snapshot-sample --evidence
```

출처 표시: `origin=live_bronze`와 `sample_found`를 본다. helper는 현재 Bronze에 보존된 r을 조회하며 새 스냅샷을 만들지 않는다. `sample_found=false`라면 뒤의 독립 fixture를 사용하고 "지금은 r의 모양을 보기 위한 예시입니다"라고 말한다. connector·offset을 초기화하지 않는다.

강사 멘트

> "아까 snapshot.mode가 initial이라고 적혀 있었죠. 그렇다고 커넥터를 다시 켤 때마다 전체 테이블을 다시 읽는 건 아닙니다. 저장된 진행 위치가 있으면 그 위치를 기준으로 이어갑니다. r은 기존 행을 스냅샷으로 읽었다는 뜻이고, c는 새 INSERT를 수집한 이벤트입니다."

학생에게 질문: "r을 새 INSERT와 똑같은 뜻으로 세면 어떤 오해가 생길까요?" 스냅샷 행을 오늘 새로 생성된 거래라고 설명하지 않도록 정리한다. Debezium 3.2 스냅샷과 이벤트.

이제 독립 파서의 입력·출력을 읽고 예시를 실행한다. 이 Python 예시는 DB·Kafka에 접속하거나 데이터를 변경하지 않는다.

```
cat labs/session02/event_parser.py
python3 labs/session02/event_parser.py --fixtures
```

코드에서 짚을 순서: `parse_event(raw_key, raw_value, metadata)` 입력 → value 전체가 null인지 분기 → JSON과 op 확인 → after 또는 before에서 PK 선택 → 수집 위치를 출력에 보존. 코드 전체를 한 줄씩 읽느라 20분을 쓰지 않는다.

강사 멘트

> "지금 보는 건 새로운 Debezium 커넥터가 아닙니다. 메시지 한 건을 받아, 우리가 확인하고 싶은 필드로 풀어보는 작은 함수입니다. 이 코드를 실행한다고 돌아가고 있는 Flink 적재 코드가 바뀌지는 않습니다."

> "생성과 수정은 after에 행이 있지만 삭제는 before를 봐야 합니다. 여기서 after만 읽으면 삭제된 ID를 놓치겠죠. 그리고 tombstone에는 이 행 객체 자체가 없으니 같은 방식으로 처리하면 안 됩니다."

> "JSON을 읽을 수 없는 입력을 만났다면 정상 이벤트처럼 값을 만들어내면 안 됩니다. 지금 함수가 어느 입력을 해석하고, 어느 입력을 따로 구분하는지 보겠습니다."

예측 후 관찰: `origin=synthetic_fixture`는 예시 입력이다. change의 `op`, `pk`, `pk_source`, before/after와 `source_lsn`을 본다. wire value가 null인 tombstone은 `value_kind=wire_null`, JSON 문자열 `null`은

`invalid_json_null`로 구분된다. `kind`와 오류 분류를 확인하고 예시 입력을 실제 Kafka 관측값으로 쓰지 않는다.

학생에게 질문: 예시를 실행하기 전에 "이 입력의 PK는 어디서 가져올까요?", "이것은 삭제 이벤트인가요, tombstone인가요?"를 묻는다. `None`은 Python 출력의 값이고 JSON에서는 `null`로 표현된다는 점도 짧게 설명한다. Python 3.11 json. 지원 이벤트와 오류 분류는 이 예제 파서의 계약이다.

마지막 3분: 학생 SQL 〔D〕에서 after와 before를 고르는 조건을 비교한다. Python 파서와 SQL이 같은 DELETE를 찾는지 묻는다. SQL은 정상 이벤트의 값 선택을 비교하는 예이며 파서의 모든 오류 검사를 재구현한 것은 아니다. 아직 실제 새 변경을 실행한 것은 아니며, 다음 20분에 방금 읽은 함수를 실제 Kafka 입력에 적용한다. Silver MERGE나 중복 제거로 확장하지 않는다.

01:15–01:35 — 새 변경 예측 후 Kafka·Bronze·파서 대조

실행 위치: 맥미니 저장소 루트, 앞에서 사용한 터미널. 이제 CASE_ID를 새로 만든다. 앞에서 읽은 1회차 키는 지우지 않는다.

```
CASE_ID=s01-s02-live-$(date +%m%d%H%M%S)
printf '새 시연 환불 ID: %s-r1\n' "$CASE_ID"
./labs/session02/lab.sh "$CASE_ID" payment
./labs/session02/lab.sh "$CASE_ID" wait --event payment --evidence
```

강사 멘트

> "앞에서는 지난 수업의 이력을 다시 읽었습니다. 지금부터는 새 키로 실제 변경을 만들겠습니다. 화면의 환불 ID가 바뀌었으니 SQL에서 비교할 값도 이 ID를 기준으로 봐주세요. 주문과 결제는 환불 시나리오를 위한 준비 데이터입니다."

01:17–01:22 — INSERT 20,000원

먼저 질문: "원천 행 수, op, before와 after 금액은 각각 어떻게 될까요?" 답을 받은 뒤 실행한다.

```
./labs/session02/lab.sh "$CASE_ID" refund
./labs/session02/lab.sh "$CASE_ID" source-rows --evidence
./labs/session02/lab.sh "$CASE_ID" wait --event refund --evidence
./labs/session02/lab.sh "$CASE_ID" kafka --summary --evidence
./labs/session02/lab.sh "$CASE_ID" bronze --summary --evidence
./labs/session02/lab.sh "$CASE_ID" parse --summary --evidence
```

예상 관찰: 원천 1행·20000, op=c, before=null, after 금액 20000. 실제 이벤트 파싱 결과의 `origin=live_kafka`, `kind`, `op`, `pk`, `pk_source`, metadata를 앞의 fixture와 구분한다.

> "이번에는 예시 JSON이 아니라 방금 Kafka에서 읽은 메시지입니다. key의 환불 ID, value의 op와 금액, Bronze의 같은 수집 위치, 파서가 고른 PK가 서로 맞는지 보세요."

01:22–01:27 — UPDATE 30,000원

먼저 질문: “같은 PK를 수정하면 원천은 두 행이 될까요? Bronze는 어떻게 달라질까요?”

```
./labs/session02/lab.sh "$CASE_ID" correction
./labs/session02/lab.sh "$CASE_ID" source-rows --evidence
./labs/session02/lab.sh "$CASE_ID" wait --event correction --evidence
./labs/session02/lab.sh "$CASE_ID" kafka --summary --evidence
./labs/session02/lab.sh "$CASE_ID" bronze --summary --evidence
./labs/session02/lab.sh "$CASE_ID" parse --summary --evidence
```

예상 관찰: 원천은 여전히 1행·30000, op=u, before 20000 / after 30000. Bronze에는 이번 키의 c와 u가 남는다. PK는 같지만 Kafka 수집 위치는 다르다. 순서는 같은 토픽·파티션 안에서 비교한다.

> “같은 PK가 다시 나온 것만으로 중복 수집이라고 할 수는 없겠죠. 원천에서는 한 행을 수정했고, 여기에는 서로 다른 변경 이벤트가 쌓인 것입니다. source.lsn과 토픽·파티션·오프셋을 따로 보는 이유도 여기에 있습니다.”

01:27–01:33 — DELETE

먼저 질문: “after가 null이면 파서는 어디서 PK를 찾을까요? value 전체가 null인 메시지도 같은 출력이 될까요?”

```
./labs/session02/lab.sh "$CASE_ID" delete
./labs/session02/lab.sh "$CASE_ID" source-rows --evidence
./labs/session02/lab.sh "$CASE_ID" wait --event delete --evidence
./labs/session02/lab.sh "$CASE_ID" kafka --summary --evidence
./labs/session02/lab.sh "$CASE_ID" bronze --summary --evidence
./labs/session02/lab.sh "$CASE_ID" parse --summary --evidence
```

예상 관찰: 원천 0행, d는 before 금액 30000 / after null. tombstone은 key가 같고 value 전체가 null이다. Bronze는 null tombstone을 제외하므로 이번 환불의 c/u/d 세 이력을 본다. 파서에서는 d의 업무 PK와 tombstone 분기가 어떻게 표현됐는지 각각 읽는다.

> “DELETE 객체 안의 after가 null인 것과 value 전체가 null인 메시지는 다른 입력입니다. 전자는 삭제 이력이고 후자는 tombstone이죠. 같은 함수로 읽어도 이 둘을 구별해서 내보내야 합니다.”

01:33–01:35 — 대조 결과 정리

```
./labs/session02/lab.sh "$CASE_ID" collect-check --stage delete --summary --evidence
```

> “예측과 달랐던 부분이 하나 있었나요? 같은 원천 키의 c/u/d가 실제 Kafka와 Bronze에 남고, 파서가 각 입력을 구별하는지까지 확인했습니다. 지금 검사는 이번 합성 키의 시나리오를 대조한 것이고 모든 장애에서의 수집 보장을 증명한 것은 아닙니다.”

공식 근거: Debezium 3.2 key/value·DELETE·tombstone, Kafka 3.9 파티션과 순서.

막히면: wait timeout 또는 조회 오류가 난 구간과 마지막으로 확인한 결과를 남긴다. 해당 단계가 도착한 것처럼 다음 기대값을 읽지 않는다. 5분을 넘기면 키·실행 시각이 있는 리허설 증거로 전환하고 01:35 진단 시간에 이 상황을 연결한다.

01:35–01:50 — 수집이 막혔다면 어디부터 볼까

실제 장애를 만들지 않는다. 아래 상황을 하나씩 읽고 학생이 다음 확인 지점을 설명하게 한다.

학생에게 보여줄 상황	강사가 연결할 근거
“원천은 바뀌었는데 Kafka에 이번 키가 없습니다.”	commit 여부, publication·include list, connector/task 상태, 조회 범위
“Kafka에는 있는데 Bronze에서는 안 보입니다.”	Flink 구독 범위·처리·checkpoint, sink commit, 키를 찾는 SQL
“Bronze에 생성·수정은 있는데 삭제가 안 보입니다.”	after만 검색했는지, 실제 key·수집 위치, d와 tombstone 구분

필요한 결과만 다시 보여준다. PG 조회는 문장별 10초, Kafka 보존 로그 스캔은 30초, runner·물리 파일 subprocess는 각각 최대 120초로 제한된다. timeout이면 미확인 구간으로 남기며 빈 결과나 성공으로 바꾸어 설명하지 않는다.

```
./labs/session02/lab.sh source-config --evidence
./labs/session02/lab.sh connector-sanitized --evidence
./labs/session02/lab.sh "$CASE_ID" kafka --summary --evidence
./labs/session02/lab.sh "$CASE_ID" bronze --summary --evidence
```

강사 멘트

> “바로 재시작하기 전에 어디까지 도착했는지부터 나눠보겠습니다. 원천, Kafka, Bronze 중 마지막으로 실제 확인한 곳이 어디인가요? 그다음 구간에서 무엇을 확인할지 말해보세요.”

> “RUNNING이나 checkpoint 완료는 상태를 보는 단서입니다. 이번 환불이 목적지에서 조회되는지까지는 별도로 확인해야 합니다.”

공식 근거: Kafka Connect connector·task 상태, Iceberg Flink commit.

01:50–02:00 — 정리·복습·다음 시간

강사 멘트

> “오늘은 삭제된 환불의 세 이력을 SELECT로 보고, 실제 파일 내용까지 연결했습니다. 그리고 원천에서 무엇을 내보내는지, Debezium이 어떻게 읽는지, Kafka 이벤트를 어떻게 해석할지 나눠서 봤습니다.”

> “개인 환경에서는 공통 예제를 다시 실행하고, 본인 SQL과 파서가 같은 삭제 이벤트를 찾는지 비교해보세요. 실행이 막혔다면 어디까지 확인했고 어느 결과가 안 보였는지 함께 질문해주시면 됩니다. 확인하지 못한 값은 추정해서 채우지 않으셔도 됩니다.”

> “다음 시간에는 Flink에서 Iceberg Bronze로 적재되는 과정과 checkpoint·commit을 더 자세히 다룹니다. 개인 프로젝트도 3회차에 원천 테이블 설계를 확정할 예정이니 관심 있는 도메인을 생각해두세요.”

마지막 질문: “삭제된 ID는 어디서 찾았나요?”, “기존 커넥터를 재시작하면 언제나 r이 나오나요?”, “Kafka까지만 보이면 어디부터 확인할까요?” 세 가지로 마무리한다. 복습 방법·제출 기한은 별도 공지를 따르고 새 필수 과제나 마감일을 즉석에서 추가하지 않는다.

수업 후 — 화면 공유 종료 뒤

증거가 남은 경로와 사용한 CASE_ID를 확인한다. 이 프로젝트를 종료할 때만 기존 savepoint 종료 명령을 사용한다. 맥미니 VM 전체나 다른 수업 컨테이너를 끄지 않는다.

```
./scripts/stop.sh
```

수업 전 대체 예제 준비 — 원래 수업 키가 없을 때만

아래는 새 복습용 데이터 생성이며 과거 수업 이력을 복구하는 명령이 아니다. 맥미니 저장소 루트에서 수업 전에 실행한다. 기존 데이터를 삭제하지 않는다. 같은 터미널의 CASE_ID가 새 값으로 바뀌므로 이후 모든 조회도 이 키를 사용한다.

```
CASE_ID=s01-s02-live-$(date +%m%d%H%M%S)
./labs/session02/lab.sh "$CASE_ID" payment
./labs/session02/lab.sh "$CASE_ID" wait --event payment --evidence
./labs/session02/lab.sh "$CASE_ID" refund
./labs/session02/lab.sh "$CASE_ID" wait --event refund --evidence
./labs/session02/lab.sh "$CASE_ID" correction
./labs/session02/lab.sh "$CASE_ID" wait --event correction --evidence
./labs/session02/lab.sh "$CASE_ID" delete
./labs/session02/lab.sh "$CASE_ID" wait --event delete --evidence
./labs/session02/lab.sh "$CASE_ID" collect-check --stage delete --summary --evidence
./labs/session02/lab.sh "$CASE_ID" bronze --summary --evidence
./labs/session02/lab.sh "$CASE_ID" parquet --summary --evidence
```

실패한 키의 증거는 남겨두고 원인을 기록한다. 수업이 시작된 뒤 예제 준비를 길게 반복하지 않는다.